Natural Language Inference: Recurrent LLM vs. Transformer Ensemble

Serhii Tupikin | Ilya Maltsev | Julius Vander Arend ◇ Group CW2 ◇ Track A: NLI
COMP34812 Natural Language Understanding | The University of Manchester, 2025–26

Task & Dataset

Natural Language Inference (NLI). Given a *premise* p and a *hypothesis* h, decide whether h is entailed by p.
Input: sentence pair (p, h)
Output: *entailment* or *contradiction* — binary label $\{1, 0\}$
Track: A — pairwise sequence classification

NLU Shared-Task NLI corpus
no external datasets allowed

Split	Pairs
Training	24,432
Development	6,736
Test (hidden)	3,302

Solution B: RWKV-7 "Goose"

Model Overview

An attention-free, recurrent 7.2B-parameter LLM.

- **Architecture:** RWKV-7 — an advanced RNN reaching transformer-level quality without quadratic self-attention.
- **Core innovation:** the *Delta Rule* replaces softmax attention with a recurrent state update, removing the $O(n^2)$ bottleneck.
- **Complexity:** **Time:** $\mathcal{O}(n)$ **Space:** $\mathcal{O}(1)$
- **Why it fits NLI:** long-range, streaming-friendly reasoning over (p, h) pairs at a fraction of transformer inference cost.

Training Strategy: LoRA Fine-Tuning

Low-Rank Adaptation tailored for binary NLI classification.

Rank / Alpha	$r = 32$ / $\alpha = 64$
Trainable params	∼83.8M (**1.15%** of total)
Frozen params	∼7.12B
Hardware	1× RTX 5090
Training time	∼**2 h** for 4 epochs

Why LoRA? Small low-rank matrices are added into the frozen RWKV-7 model — updating only ∼1.15% of its weights — to fine-tune it for NLI while keeping its general language abilities intact.

Results on Dev Set

	F1	Accuracy	MCC
Baseline — LSTM	0.6603	0.6606	0.3206
Our RWKV-7 (LoRA)	**0.9419**	**0.9399**	**0.8796**

Key Takeaways

- **Delta rule is all you need**: the attention-free recurrent backbone matches transformer-level quality at *linear-time* inference cost.
- **Matches a heavy transformer ensemble**: despite being recurrent, RWKV-7 reached the same task-level performance as our compute-heavy Transformer Ensemble (Solution C) on this NLI task.
- **Scale vs. efficiency**: the lighter 2.9B variant came *close but not as good* even after 7 epochs (vs. 4 for 7.2B) — we chose 7.2B because the goal was maximum achievable performance, not minimum compute.

Solution C: Transformer Ensemble

Model Overview

Blending ensemble of four transformers with an XGBoost meta-learner.

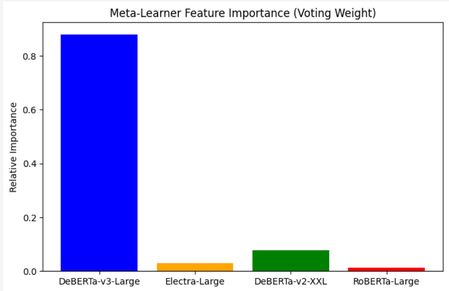
- **Base models:** DeBERTa-v3-large, ELECTRA-large, DeBERTa-v2-XXLarge, RoBERTa-large.
- **Combination:** per-model softmax probs → XGBoost classifier (100 trees, depth 5, lr 0.1). Learns optimal weighting per model.
- **Optimization:** aggressive hyper-parameter optimization in ensemble using Optuna.
- **Why:** Diverse ensemble architecture demonstrated significant performance improvements over a single model.

Training Strategy

Optuna HPO (20 trials) → FGM fine-tuning → XGBoost blending.

Model	Tuning	LR	Loss
DeBERTa-v3-L	Full FT	8.36e-6	CE
ELECTRA-L	Full FT	1.52e-5	CE
RoBERTa-L	Full FT	7.80e-6	Focal
DeBERTa-v2-XXL	LoRA $r=8$	2.65e-4	CE

Epochs	5	**Batch**	16 (accum for XXL)
Adversarial	FGM $\epsilon=1.0$	**Optimiser**	AdamW, wd 0.01
LR	linear warmup 10%	**Time**	∼2 h total



XGBoost meta-learner voting weights — DeBERTa-v3-L dominates.

Results on Dev Set

	F1	Acc.	MCC
Baseline — BERT	0.8198	0.8202	0.6400
Single best model (DeBERTa-v3-L)	0.9480	0.9480	0.8960
Our Ensemble (XGBoost)	**0.9621**	**0.9621**	**0.9242**

Key Takeaways

- ∼**1.13B trainable params** (1,127,362,370) across the four-model ensemble; DeBERTa-v2-XXL uses LoRA while the three others are full fine-tunes.
- **FGM adversarial training** improved all models' F1 in Optuna — perturbation $\epsilon=1.0$ on word embeddings.
- **XGBoost meta-learner** beats simple averaging by learning non-uniform per-model weights; ensemble F1 = 0.9621 vs. single-best ∼0.9278.

Head-to-Head Comparison (Dev)

System	Params (train)	F1	Acc.	MCC
RWKV-7 + LoRA	∼83.8M	**0.9419**	0.9399	0.8796
Transformer Ensemble	∼1.13B trainable	**0.9621**	0.9621	0.9242

A recurrent LLM with 1.15% tunable parameters competes directly with a heavy transformer ensemble — the core headline of this project. The ensemble edges ahead with F1 0.9621 vs. 0.9419, at the cost of ∼1.13B trainable params.

Limitations & Ethical Considerations

- **Architecture implementation constraints.** We use the `fla-hub` port of RWKV-7 for HF/PEFT compatibility; it does not yet fully replicate the native CUDA reference's initialisation rules and layer-specific learning rates, so performance may sit slightly below the theoretical maximum.
- **Pre-training bias.** Both systems inherit biases from web-scale pre-training corpora.
- **Closed-task caveat.** No external data used in fine-tuning, but pre-training corpora are *not* controlled or audited.
- **Label schema.** Binary entailment is a simplification — real-world inference often includes a *neutral* class.

References

Solution B — RWKV-7: [1] Hu et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models.* arXiv:2106.09685. [2] Peng et al. (2025). *RWKV-7 "Goose" with Expressive Dynamic State Evolution.* arXiv:2503.14456.
Solution C — Transformer Ensemble: [1] Chen & Guestrin (2016). *XGBoost: A Scalable Tree Boosting System.* KDD '16, pp. 785–794. [2] Clark et al. (2020). *ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators.* arXiv:2003.10555. [3] He et al. (2021). *DeBERTa: Decoding-enhanced BERT with Disentangled Attention.* arXiv:2006.03654. [4] He et al. (2023). *DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing.* arXiv:2111.09543. [5] Hu et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models.* arXiv:2106.09685. [6] Liu et al. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach.* arXiv:1907.11692.

https://github.com/d4k3r/NLU Model cards & demo notebooks in Canvas submission